SAHAYAK

CONTINUOUS SIGN COLLECTION, TRAINING AND DEPLOYMENT

A repeatable workflow for adding a movement-based sign while keeping the static-sign model unchanged.

THE SAFE WORKFLOW

1. Back up continuous samples and the active sequence model.
2. Record a complete movement sequence under one stable label.
3. Train a candidate, then test on fresh recordings.
4. Promote only after reviewing the results.
5. Deploy only sahayak_sequence.joblib, not the static model.

Sequence input size: 1320 features per complete recording.

Project folder

`C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training`

STEP 1 - BACK UP

```
Set-Location "C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training"
.\.venv\Scripts\Activate.ps1
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = ".\backups\sequence_$stamp"
New-Item -ItemType Directory -Force $backup | Out-Null
Copy-Item .\continuous_gesture_db.db $backup -Force
Copy-Item .\model\sahayak_sequence.joblib $backup -Force
```

Confirm (.venv) is active. Preserve the printed backup path.

Check existing continuous labels

```
python -c "import sqlite3; c=sqlite3.connect('continuous_gesture_db.db'); print(*c.execute('SELECT label, COUNT(*) FROM sequence_samples GROUP BY label ORDER BY label'), sep='\\n'); c.close()"
```

The source database is continuous_gesture_db.db. Do not edit or replace the static CSV or legacy gesture databases.

STEP 2 - COLLECT THE NEW SIGN

Run the sequence collector once for each label:

```
python collect_sequences.py
```

When prompted, type one stable uppercase label with underscores, for example:

```
WHAT_IS_YOUR_NAME
```

Perform the complete movement each time. Press C in the camera window to save one sequence.

Follow the on-screen quit key to finish.

How many examples?

Aim for about 30-40 varied recordings for the new sign, if possible. Change speed and position naturally while preserving the correct sign.

Repeat the collector for every separate label you intend to train. The collector asks for one label per run.

Verify sample counts

```
python -c "import sqlite3; c=sqlite3.connect('continuous_gesture_db.db'); print(*c.execute('SELECT label, COUNT(*) FROM sequence_samples GROUP BY label ORDER BY label'), sep='\\n'); c.close()"
```

Fix spelling mistakes before training: labels such as GPPD_MORNING create a separate class until corrected.

STEP 3 - TRAIN CANDIDATE

Train from the continuous database without changing the active model:

```
python train_sequence_model.py --db
```

The trainer validates samples and reports label counts, holdout accuracy and per-label metrics.

It writes a candidate model:

```
model\sahayak_sequence_candidate.joblib
```

Verify the candidate

```
python -c "import joblib; m=joblib.load(r'model\sahayak_sequence_candidate.joblib');
print('Features:',m.n_features_in_); print('Labels:',sorted(str(x) for x in m.classes_))"
```

Expect 1320 features and all intended continuous labels.

Do not use --promote yet. A high random holdout score is not a guarantee of real-world accuracy.

STEP 4 - FRESH TEST

After training the candidate but before recording test samples, record the current database ID:

```
$testStartId = [int](python -c "import sqlite3; c=sqlite3.connect('continuous_gesture_db.db';
print(c.execute('SELECT COALESCE(MAX(id),0) FROM sequence_samples').fetchone()[0]); c.close()")
```

Keep this terminal open so $testStartId is preserved.

Record at least five NEW examples per label with collect_sequences.py. Do not retrain before testing. These rows must have IDs above $testStartId.

Create the reusable test script

```
@'
import argparse, json, sqlite3, joblib, numpy as np
from sequence_features import summarize_sequence
p=argparse.ArgumentParser(); p.add_argument('--after-id',type=int,required=True); a=p.parse_args()
m=joblib.load('model/sahayak_sequence_candidate.joblib')
c=sqlite3.connect('continuous_gesture_db.db')
r=c.execute('SELECT id,label,frames_json FROM sequence_samples WHERE id>? ORDER BY
id',(a.after_id,)).fetchall(); c.close()
if not r: raise SystemExit('No fresh test samples found')
X=np.asarray([summarize_sequence(json.loads(z)) for _,_,z in r]); y=np.asarray([lab for _,lab,_ in r])
pred=m.predict(X); print('Samples:',len(r),'Accuracy:',round(float(np.mean(y==pred))*100,1),'%')
for lab in sorted(set(y)): print(lab,int(np.sum((y==lab)&(pred==y))), '/', int(np.sum(y==lab)))
'@ | Set-Content .\test_sequence_candidate.py -Encoding utf8
```

Run the fresh test

```
python .\test_sequence_candidate.py --after-id $testStartId
```

The candidate must not be trained on these fresh recordings before the test.

STEP 5 - PROMOTE AND VERIFY

Review fresh-test results. If a class performs poorly, collect more varied examples and investigate before activation.

When satisfied, run:

```
python train_sequence_model.py --db --promote
```

This retrains using the current full database and backs up the prior active sequence model.

Verify the active model

```
python -c "import joblib; m=joblib.load(r'model\sahayak_sequence.joblib');
print('Features:',m.n_features_in_); print('Labels:',sorted(str(x) for x in m.classes_))"
```

Then run the automated tests:

```
python -m unittest test_sahayak.py test_sequence.py test_sequence_db.py
```

Start the local API with python web_app.py and test each old and new continuous sign in the local Studio.

If the new model causes regressions, restore the previous sequence-model backup. Do not change the static model.

STEP 6 - DEPLOY THE SEQUENCE MODEL

Copy only the promoted sequence model into the deployment project:
`.\deployment\SAHAYAK-Vercel-Deployment\backend\model\sahayak_sequence.joblib`
Commit this file to the branch watched by Render and wait for the deployment to complete.

Verify the deployed backend

`https://sahayak-backend-krh1.onrender.com/api/status`
Confirm sequence_model_ready is true and sequence_sign_labels lists the expected continuous signs.
Open the live Studio and test old and new sequences. If the status endpoint lists old labels, check the path, commit, and Render deployment.

Do not deploy training data

Do not replace sahayak_twohand.joblib for a continuous-only change. Do not upload continuous_gesture_db.db, the static CSV, or raw samples to the website repository.
Only publish the trained model artifact needed by the backend. Keep backups and consented training samples private.

Repeatable checklist

[ ] Backup. [ ] Label spelling verified. [ ] Candidate reviewed. [ ] Fresh test passed.
[ ] Promotion and regression tests passed. [ ] Sequence model deployed. [ ] Live Studio checked.

COPY/PASTE COMMANDS - CONTINUOUS SIGNS (1/4)

Run these blocks in VS Code PowerShell. Paste one shaded block at a time and wait for it to finish.
The path matches this guide; change only the first line if your folder is elsewhere.

1. Open the project and activate Python

```
$ProjectRoot = "C:\Users\user\Videos\SAHAYAK-Updated-UI-Local-Training"
if (-not (Test-Path $ProjectRoot)) { throw 'Edit $ProjectRoot to the folder on this PC.' }
Set-Location $ProjectRoot
if (-not (Test-Path ".\.venv\Scripts\Activate.ps1")) { throw ".venv is missing." }
.\.venv\Scripts\Activate.ps1
python --version
python -c "import sys; print(sys.executable)"
```

2. Back up the continuous-sign database and active model

```
if (-not (Test-Path ".\continuous_gesture_db.db")) { throw "Database missing; stop and confirm folder." }
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = ".\backups\sequence_$stamp"
New-Item -ItemType Directory -Force $backup | Out-Null
Copy-Item ".\continuous_gesture_db.db" $backup -ErrorAction Stop
if (Test-Path ".\model\sahayak_sequence.joblib") { Copy-Item ".\model\sahayak_sequence.joblib" $backup -Force }
Write-Host "Backup saved to $backup"
```

3. Print existing labels and sample counts

```
@'
import sqlite3
from pathlib import Path
path = Path("continuous_gesture_db.db")
if not path.exists():
    raise SystemExit("Database not found; check the project folder.")
with sqlite3.connect(path) as db:
    rows = db.execute(
        "SELECT label, COUNT(*) FROM sequence_samples GROUP BY label ORDER BY label"
    ).fetchall()
print("Total sequences:", sum(count for _, count in rows))
for label, count in rows:
    print(f"{label}: {count}")
'@ | python -
```

If the database or table is missing, stop here. Do not create a new database by guessing its location.

COPY/PASTE COMMANDS - CONTINUOUS SIGNS (2/4)

Keep the same PowerShell terminal open so the project folder and active .venv stay selected.

4. Collect the new movement-based sign

```
python collect_sequences.py
```

Run the collector once per label. Type one stable uppercase label such as WHAT_IS_YOUR_NAME. Perform the full movement and press C in the camera window to save a sequence. Aim for 30-40 varied examples per label when possible.

5. Recheck counts after collection

```
@'
import sqlite3
with sqlite3.connect("continuous_gesture_db.db") as db:
    rows = db.execute(
        "SELECT label, COUNT(*) FROM sequence_samples GROUP BY label ORDER BY label"
    ).fetchall()
print("Total sequences:", sum(count for _, count in rows))
for label, count in rows:
    print(f"{label}: {count}")
'@ | python -
```

6. Train a candidate; this does not promote it

```
python train_sequence_model.py --db
```

7. Confirm candidate feature count and labels

```
@'
import joblib
model = joblib.load(r"model\sahayak_sequence_candidate.joblib")
print("Features:", model.n_features_in_)
print("Labels:", sorted(str(x) for x in model.classes_))
'@ | python -
```

Expect 1320 features and every intended continuous-sign label. Do not promote if a class is missing or an accidental label appears.

COPY/PASTE COMMANDS - CONTINUOUS SIGNS (3/4)

Fresh testing must use recordings that were not used to train the candidate. Keep this same PowerShell terminal open.

8. Record the database ID before collecting test samples

```
$rawId = @'
import sqlite3
with sqlite3.connect("continuous_gesture_db.db") as db:
    print(db.execute("SELECT COALESCE(MAX(id), 0) FROM sequence_samples").fetchone()[0])
'@ | python -
$testStartId = [int]$rawId.Trim()
Write-Host "Fresh test IDs must be greater than $testStartId"
```

9. Record at least five fresh examples per label

```
python collect_sequences.py
```

Collect at least five NEW examples for every intended label after running the ID command. Do not retrain the candidate before testing these new rows.

10. Create the fresh-test script (paste the complete block)

```
@'
import argparse
import json
import sqlite3
import joblib
import numpy as np
from sequence_features import summarize_sequence

parser = argparse.ArgumentParser()
parser.add_argument("--after-id", type=int, required=True)
args = parser.parse_args()
model = joblib.load("model/sahayak_sequence_candidate.joblib")
with sqlite3.connect("continuous_gesture_db.db") as db:
    rows = db.execute(
        "SELECT id, label, frames_json FROM sequence_samples WHERE id > ? ORDER BY id",
        (args.after_id,),
    ).fetchall()
if not rows:
    raise SystemExit("No fresh test samples found.")
X = np.asarray([summarize_sequence(json.loads(frames)) for _, _, frames in rows])
y = np.asarray([label for _, label, _ in rows])
pred = model.predict(X)
print("Fresh samples:", len(rows))
print("Accuracy:", round(float(np.mean(y == pred)) * 100, 1), "%")
for label in sorted(set(y)):
    mask = y == label
    correct = int(np.sum((pred == y) & mask))
    print(f"{label}: {correct}/{int(np.sum(mask))} correct")
'@ | Set-Content ".\test_sequence_candidate.py" -Encoding utf8
```

This block saves test_sequence_candidate.py in the project folder. Run it after all fresh test recordings have been saved.

COPY/PASTE COMMANDS - CONTINUOUS SIGNS (4/4)

11. Test the candidate on the fresh recordings

```
python .\test_sequence_candidate.py --after-id $testStartId
```

Review the overall accuracy and each label's correct/total count. If any label performs poorly, collect more varied examples, retrain the candidate, capture a new fresh test set, and test again.

12. Promote only after the fresh test is acceptable

```
python train_sequence_model.py --db --promote
```

13. Run regression tests

```
python -m unittest test_sahayak.py test_sequence.py test_sequence_db.py
```

14. Verify the promoted model

```
@'
import joblib
model = joblib.load(r"model\sahayak_sequence.joblib")
print("Features:", model.n_features_in_)
print("Labels:", sorted(str(x) for x in model.classes_))
'@ | python -
```

15. Copy only the promoted sequence model to the deployment project

```
$deployModel = ".\deployment\SAHAYAK-Vercel-Deployment\backend\model\sahayak_sequence.joblib"
if (-not (Test-Path (Split-Path $deployModel))) { throw "Deployment model folder not found; confirm its path." }
Copy-Item ".\model\sahayak_sequence.joblib" $deployModel -Force
Write-Host "Copied model to $deployModel"
```

16. Check the live backend after the Render deployment finishes

```
Invoke-RestMethod "https://sahayak-backend-krh1.onrender.com/api/status" | ConvertTo-Json -Depth 5
```

Confirm sequence_model_ready is true and sequence_sign_labels includes the expected signs. Never upload the training database or raw sequence samples.